

WG01: Kernel C++: A Methodology for Freestanding Development

Amlal El Mahrouss

December 2025

Abstract

Many Operating Systems Kernels have been shipped using the C programming language. And some of them like EKA2 use the C++ programming language. Although notoriously difficult, one may still adapt to those constraints in order to deliver one such operating system kernel. That is the reason that most production-grade kernels (Linux, XNU, and NT) are mostly written in C. With a higher-level subset in C++. However, when correctly applying C++ principles to kernel development, one makes the development much more easier to pull off.

1 The Three Principles of Kernel C++.

1.1 Part One: The C++ Runtime.

The problem mostly lies in the C++ runtime. Which assumes an existing host. A host is the contrary of a freestanding target, that is a program which expects a runtime to be present and linked to the program. A C++ Kernel may instead make use of compile-time features of C++ alongside a tiny C++ runtime to make sure that no issues arise because of this host/freestanding difference.

1.2 Part Two: constexpr and Friends.

One may avoid V-tables or runtime when possible. While focusing instead on meta-programming and compile-time features offered by C++. For example one may use templates to implement a scheduling policy algorithm. One example of such implementation may be:

```
1  /// Reference Implementation: https://github.com
2  /// /nekernel-org/nekernel/blob/stable/src/kernel/
3  /// KernelKit/CoreProcessScheduler.h#L51-L76
4  /// AND: https://github.com/nekernel-org/nekernel/blob/stable/src/kernel
5  /// /KernelKit/CoreProcessScheduler.h#L78-L105
6
7  struct FileTree final {
8      static constexpr bool is_virtual_memory = false;
9      static constexpr bool is_memory = false;
10     static constexpr bool is_file = true;
11     /// ...
12 };
13
14 struct MemoryTree final {
15     static constexpr bool is_virtual_memory = false;
16     static constexpr bool is_memory = true;
17     static constexpr bool is_file = false;
18     /// ...
19 };
```

As you see, two structures leverages the ‘constexpr’ keyword to make sure no bugs or panic occur at runtime because of a misuse of a system resource.

Which is why the constexpr keyword is very powerful here, we avoid the many pitfalls of writing (and hoping) that the C version will be well-thought enough so that we can catch such bugs later.

1.3 Bonus: Using Concepts and the DDK.

This bonus subsection intends to show how properly applied C++ can solve issues related to driver validation. The following example shows how powerful C++ can be when combining it with Device Driver development as well.

```
1  /// Reference implementation:
2  /// https://github.com/nekernel-org
3  /// /nekernel/blob/stable/src/libDDK/DriverKit
4  /// /c%2B%2B/driver_base.h
5  /// @brief This concept requires the driver
6  /// to be IDriverBase compliant.
7
8  inline constexpr auto kInvalidType = 0;
9
10 template <class Driver>
11 concept IsValidDriver = requires(Driver drv) {
12     { drv.IsActive() && drv.Type() > kInvalidType };
13 };
14
15 /// @brief Consteval helper to detect
16 /// whether a template is truly based on IDriverBase.
17 /// @note This helper is consteval only.
18 template<IsValidDriver T>
19 consteval void ce_ddk_is_valid(T) {}
```

1.4 Part Three: Memory and C++ Classes.

This last part treats about the final and most important part of this paper so far. Memory. As you may already have known, the C++ language uses a class lookup system (also called v-table) in order to refer to a base method in case if the instance has it missing.

```
1  /// Link: https://godbolt.org/z/aK6Y98xnd
2  /// /std:c++20 /Wall
3
4  #include <iostream>
5
6  class A
7  {
8  public:
9      explicit A() = default;
10     virtual ~A() = default;
11
12     virtual void doImpl()
13     {
14         std::cout << "doImpl()\r\n";
15     }
16 };
17
18 class B : public A
19 {
20 public:
21     explicit B() = default;
22     ~B() override = default;
23 };
24
25 int main() {
26     B callImpl;
27     callImpl.doImpl();
```

2 Addressing V-Tables in a C++ Kernel.

Now the problem with kernel development is that we want to avoid such feature as much as possible, and we'd do that by following the Prong on Inheritance:

2.1 The Three Prongs on Inheritance Decision Framework.

TTPI is a thought exercise used to decide whether you should consider using C++ in a kernel. Consider the following thought exercise:

- 1: Is this a feature that can be implemented with other similar protocols/concepts?
- 2: Is this doable without too much trade-off costs?
- 3: Is this doable without V-Tables?

If 2/3 of those questions fail, you should consider finding another solution to your problem. As it surely has an equivalent without the problematic aspects.

2.2 The Vetable Pattern, validating containers for Kernel C++.

The following concept makes sure that the 'class T' is truly vettable by the Operating System Kernel. Such properties are called 'Vetable' in a C++ kernel.

```
1 template <class T, typename OnFallback>
2 concept IsVetable = requires(OnFallback fallback) {
3     { Vetable<T>::kValue ? true : fallback() };
4 };
```

The vettable structure makes use of template metaprogramming in modern C++ in order to evaluate whether a container is vettable or not, such containers inherits the 'IVetable' structure and are marked final.

```
1 struct IVetable {
2     explicit IVetable() = default;
3     virtual ~IVetable() = default;
4
5     NE_COPY_DEFAULT(IVetable)
6 };
```

A vettable system may look as such in a Kernel C++ system.

```
1 #define NE_VETTABLE final : public ::Kernel::IVetable
2 #define NE_NOT_VETTABLE final : public ::Kernel::INotVetable
3
4 namespace Kernel {
5     /// @brief Vet interface for objects.
6     struct IVetable {
7         explicit IVetable() = default;
8         virtual ~IVetable() = default;
9
10        NE_COPY_DEFAULT(IVetable)
11    };
12
13    struct INotVetable {
14        explicit INotVetable() = default;
15        virtual ~INotVetable() = default;
16
17        NE_COPY_DEFAULT(INotVetable)
18    };
19
20    template <class Type>
21    struct Vetable final {
```

```

22     static constexpr bool kValue = false;
23 };
24
25 template <>
26 struct Vetable<INotVetable> final {
27     static constexpr bool kValue = false;
28 };
29
30 template <>
31 struct Vetable<IVetable> final {
32     static constexpr bool kValue = true;
33 };
34
35 /// @brief Concept version of Vetable.
36 template <typename T, typename OnFallback>
37 concept IsVetable = requires(OnFallback fallback) {
38     { Vetable<T>::kValue ? true : fallback() };
39 };
40
41 template <class Type, typename OnFallback>
42 concept IsNotVetable = requires(OnFallback fallback) {
43     { !Vetable<Type>::kValue ? true : fallback() };
44 };
45 } // namespace Kernel

```

Source: <https://github.com/nekernel-org/nekernel/blob/stable/src/kernel/NeKit/Vetable.h>

3 Conclusion

We can now conclude that C++ in Kernel Development is indeed possible under a subset of C++, here called Kernel C++. A reference implementation of this paper exists, It's called NeKernel.org, available under the same internet address. We are looking forward to any questions or inquiries at: contact@nekernel.org.

4 References

1. NeKernel.org (2025). NeKernel Operating System. Available at: <https://nekernel.org>
2. Sales, J., Tasker, M. (2005). Introducing EKA2. Symbian OS Internals. Wiley. Available at: https://media.wiley.com/product_data/excerpt/47/04700252/0470025247.pdf
3. Driesen, K., Hölzle, U. (1996). The direct cost of virtual function calls in C++. *OOPSLA '96*. ACM. DOI: 10.1145/236338.236369